

GRAPH-GUIDED LONG-HORIZON MPC WITH LEWORLDMODEL

Anonymous authors

Paper under double-blind review

ABSTRACT

The cross-entropy method (CEM) planner used with LeWorldModel (LeWM) ranks candidate action sequences by the Euclidean distance between a predicted terminal latent and a goal latent. While this objective succeeds on short-horizon goals, its performance degrades sharply as goals become more temporally distant or come from a different episode. On Push-T, the planner reaches same-episode goals 25, 50 and 100 steps ahead, and cross-episode goals, with success rates of 91/41/13/14%, respectively. We address this degradation at long horizons and across episodes without retraining the world model or learning a subgoal generator. We learn a temporal distance representation over frozen latents, build a graph of the offline dataset, and select a subgoal along the graph’s shortest path. A reachability critic scores predicted endpoints by expected time to goal, and latent L2 is used for the final horizon. Each component addresses a distinct limitation: the subgoal provides *direction*, the critic accounts for *reachability*, and the switch to L2 restores near-goal *resolution*. The graph finds a route for every evaluated cross-episode pair, but reaching the goal still requires the critic. Across goals at offsets 50 and 100, and on cross-episode goals, our method raises success from 41/13/14% to 70/46/42% on Push-T, from 91/83/29% to 98/98/66% on Reacher, and from 34/36/16% to 50/50/28% on Cube. At the 25-step horizon, our method trails L2 by 2.8 points on Push-T (88.4 vs 91.2) and 1.2 points on Reacher (76.0 vs 77.2), but leads by 6.0 points on Cube (62.0 vs 56.0).

1 INTRODUCTION

Joint-embedding world models trained from pixels (???) plan by search: they sample candidate action sequences, roll each through a learned latent predictor, and select the sequence whose predicted terminal latent lies closest to the goal latent in Euclidean (L2) distance – the default objective we call L2 throughout. On the horizons these models are typically evaluated on, where the goal is a frame from the same trajectory a fixed number of steps ahead, this objective performs well. Its performance degrades sharply, however, as the goal becomes more temporally distant. With the frozen LeWorldModel (LeWM) checkpoint and its published cross-entropy-method (CEM) planner, success on Push-T drops steadily as the goal moves further ahead in time, and for goals drawn from a different episode. Giving the planner a larger step budget does not recover this loss (table 3b). In section 5 we show that the failure lies in the objective itself, not just in the predictor.

Recent work targets this failure, mostly on a frozen LeWM-style model, by learning something new on top of it (???????). An older line of work in offline goal-conditioned reinforcement learning instead searches the data directly, letting shortest paths over a dataset graph select subgoals (?????), but executes them with a learned policy or a model trained for the purpose (section 2). To our knowledge, no prior method gives this graph to a frozen, pretrained pixel world model through its planning cost alone. ? point to why that would help: subgoals must remain “data-supported, executable, and matched to the temporal scale” to transfer to a compact frozen world model.

We place the dataset graph inside LeWM’s planner: a subgoal that is data-supported, executable, and matched to the temporal scale by construction. We learn a temporal distance representation (TDR) on the frozen LeWM latents (??), build a graph over the offline dataset in that space, and replace LeWM’s terminal cost with the distance to a subgoal – a dataset frame reached from the current state by the shortest path of observed transitions toward the goal, truncated to one planning horizon. This

alone meaningfully improves the mid-range regime, but does nothing for cross-episode goals: the graph finds a route for every pair we evaluate, well inside the planning budget, yet the planner does not execute it. The missing component is reachability, which we address with a critic $V(z, z_g, h)$, the probability that the goal is reachable from latent z within h steps, and score each predicted endpoint by *expected hitting time* under the steps remaining. With this term, and with the final horizon handed back to LeWM’s own objective once the goal is within one horizon, a single configuration matches or exceeds every alternative we tested at offsets 50 and above and on cross-episode goals, across three environments – Push-T, Reacher, and Cube (table 1).

Our contributions are:

1. **Dataset-graph subgoals and budget-clocked reachability inside an unmodified planner.** Subgoals are dataset frames on a shortest path over the offline data, enabling cross-episode stitching; a critic scores endpoints by expected hitting time within the remaining budget. Without changing the world model or CEM, this improves success beyond one horizon and on cross-episode goals in all three environments (section 3 and table 1).
2. **A cross-episode evaluation for frozen pixel world-model planners.** We evaluate goals drawn from a different episode, on fixed task pools for Push-T, Reacher and Cube. LeWM’s planner has poor performance on such goals, and budget beyond 100 steps adds almost nothing (section 4.1 and tables 1 and 3).
3. **A component analysis.** On Push-T the graph, the critic and the final switch address different failures (direction, reachability, near-goal resolution), which we support with oracle rank correlations, hazard rates and near-miss analysis (sections 4.3 and 4.4).

2 RELATED WORK

Planning with latent world models. PlaNet and Dreamer (??) plan or learn policies in a reconstruction-based latent. DINO-WM (?), V-JEPA 2-AC (?) and LeWM (?) instead plan zero-shot with CEM in a joint-embedding latent, scoring candidates by terminal latent distance to the goal; PLDM (?) uses MPPI with a cost summed over the horizon. ? study which design choices matter for these planners.

Repairing the cost on a frozen model. TRM (?) learns a pairwise temporal cost from logged trajectories and uses it as the terminal ranking on a frozen model; other methods retrain the encoder so that latent distance better tracks task progress (?????). Our TDR is in the same family, but we use it to build a graph and pick a data-supported target rather than as the cost, and we show the metric alone is not what helps (section 4.3). RC-aux’s budget-conditioned reachability (?) is the closest relative of our critic; ours is a separate network with an explicit plan clock and never touches the encoder.

Hierarchical planning with world models. HWM (?), SAGE (?), FF-JEPA (?) and Hi-LeWM (?) add a learned high-level model (coarser world models, a subgoal generator with an action prior, an action-free planner, or macro-actions), and Hi-LeWM shows that CEM can exploit such a learned subgoal space. Each trains on same-trajectory windows, so its subgoals are not built to lie on a route that no single trajectory took; ours are dataset frames on a shortest path over the whole dataset, and the cross-episode protocol is where that difference shows. SAGE reports stronger same-episode numbers than ours at long offsets (64.7% at $H = 150$ on Push-T, against 12.7% for LeWM alone); see section 6.

Graph search over offline data. SPTM (?), SoRB (?) and SGM (?) build graphs over stored experience with a learned distance, GAS (?) clusters a temporal distance representation (?) into nodes with a temporal-efficiency filter, and TTGS (?) runs the search at test time around frozen policies; all hand the subgoal to a learned goal-conditioned policy. ALPS (?), the closest prior work, reaches Laplacian cluster-graph subgoals with CEM, but over a forward model trained for the method and seeded by a learned behaviour prior that its results depend on. We take GAS’s graph construction essentially unchanged and hand the subgoal to a frozen, publicly released pixel world model through its terminal cost only, with no retrained dynamics or learned action proposals.

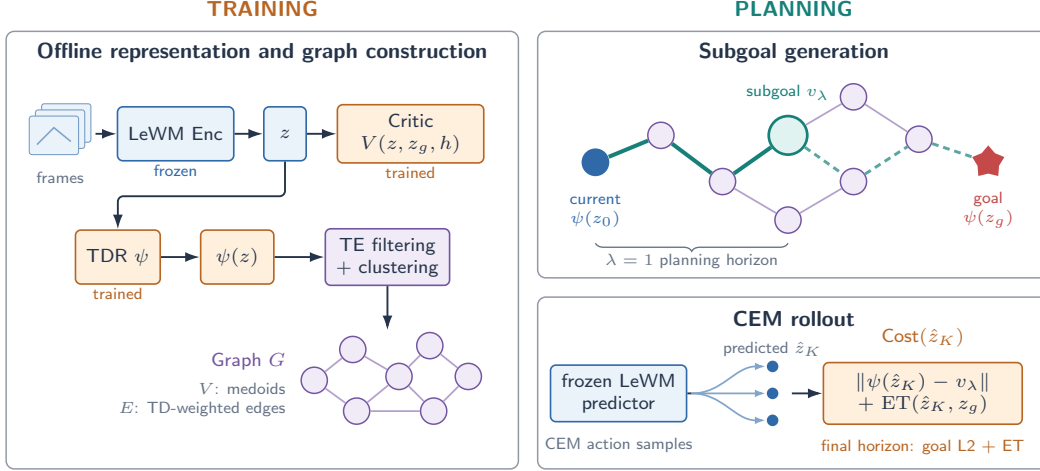


Figure 1: Graph-guided latent MPC. Training (left): We train a Critic: V , TDR ψ and graph G from the dataset frames. Planning (right): Graph identifies a subgoal. CEM scores the candidate rollouts

Value functions inside MPC. POLO (?) and TD-MPC (??) put a reward-trained value at the end of a short model rollout, and goal-conditioned terminal values have been used for multi-task MPC (?). Our critic is instead a reachability classifier trained from hindsight and cross-episode negatives, and it enters the cost as a budget-aware expected hitting time rather than a value.

3 METHOD

Frozen world model and CEM planner. LeWM (?) encodes a 224×224 frame into a 192-d latent z with a ViT encoder E ; its predictor P maps three past latents and three action blocks (five environment actions each, frame-skip 5) to the next latent. Planning is CEM over $K = 5$ blocks (25 environment steps) with 300 candidates, 30 iterations and 30 elites (LeWM’s released configuration), scoring each candidate by the terminal cost $c_{L2} = \|\hat{z}_K - z_g\|^2$ to the encoded goal frame. The optimum is executed for 25 steps and the planner is called again. We keep E , P and CEM fixed and change only the cost that scores $(z_0, \hat{z}_{1..K}, z_g)$. Beyond one horizon, L2 loses its ranking signal (section 5), so there we substitute three signals that stay informative at range: TDR distance (section 3.1), a subgoal on a dataset graph (section 3.2), and a reachability critic (section 3.3). Within one horizon we keep L2 (figure 1).

3.1 TEMPORAL DISTANCE REPRESENTATION AND DATASET GRAPH

We train a temporal distance representation $\psi : \mathbb{R}^{192} \rightarrow \mathbb{R}^{32}$ on the encoded dataset $\mathcal{D} = \{(z, z')\}$ of consecutive frames by expectile TD, as in HILP and GAS (??). With $V(z, z_g) = -\|\psi(z) - \psi(z_g)\|$, an EMA target $\bar{V}^-$, and TD target $q = -1 + \gamma \bar{V}^-(z', z_g)$ with $\gamma = 0.99$,

$$\mathcal{L}_{\text{TDR}} = \mathbb{E}_{(z, z') \sim \mathcal{D}, z_g} [\ell_{0.99}(q - V(z, z_g))], \quad \ell_\tau(u) = |\tau - \mathbf{1}_{u < 0}| u^2, \quad (1)$$

with goals relabelled as in GAS (section H). $\|\psi(z) - \psi(z_g)\|$ then approximates the discounted step count between z and z_g ; we convert TDR distance to steps with $u_k = \text{median}_t \|\psi(z_t) - \psi(z_{t+k})\|$ over same-episode pairs k steps apart.

The graph follows GAS. Let $F(z_t)$ be the first later frame of t ’s trajectory with $\|\psi(F(z_t)) - \psi(z_t)\| \geq H_{\text{TD}}$ (the episode’s last frame if none), and define temporal efficiency

$$\text{TE}(z_t) = \cos \angle(\psi(F(z_t)) - \psi(z_t), \psi(z_{t+n_{\text{TD}}}) - \psi(z_t)), \quad (2)$$

the agreement between where the logged trajectory moves in $n_{\text{TD}} = \lfloor H_{\text{TD}} \rfloor$ environment steps (H_{TD} rounded to a step count, as in GAS) and the direction ψ calls optimal. Frames with $\text{TE} <$

$\theta_{TE} = 0.9$ are dropped; the remainder are clustered sequentially in ψ -space – assign to the nearest existing centre if within $H_{TD}/2$, else open a new one, then reset centres to member means – giving nodes $v \in V_G$ with centre $c_v \in \mathbb{R}^{32}$, the mean ψ of the node’s members. Nodes within H_{TD} of each other are joined by an edge weighted by their ψ -distance, with $H_{TD} = u_{12}$. At planning time the goal is attached as a temporary node linked to every node within $\max(H_{TD}, 1.2 d_{\min})$ ($d_{\min}$ is the goal’s distance to its nearest node), and one Dijkstra pass gives the cost-to-go $D_g[v]$ from every node.

3.2 SUBGOAL ONE HORIZON AHEAD, AND THE FINAL SWITCH

At each planning call with current latent z_0 , we select the start node

$$v^* = \arg \min_{v: \|\psi(z_0) - c_v\| \leq H_{TD}} D_g[v] + \|\psi(z_0) - c_v\|, \quad (3)$$

and walk the shortest path from v^* toward the goal until the cumulative ψ -distance from z_0 first reaches the lookahead $\lambda = u_{25}$, the calibrated distance of one planning horizon; call the node reached v_λ . The base cost is

$$c_{\text{base}}(\hat{z}_K) = \begin{cases} c_{L2}(\hat{z}_K) & \text{if } \|\psi(z_0) - \psi(z_g)\| \leq \lambda \text{ or the walk reaches } z_g, \\ \|\psi(\hat{z}_K) - c_{v_\lambda}\| =: c_{\text{sub}}(\hat{z}_K) & \text{otherwise;} \end{cases} \quad (4)$$

we call the first case the *final switch*. Section 4.3 shows that a one-horizon lookahead beats shorter ones and beats tracking the intermediate latents $\hat{z}_{1..K-1}$, and that the switch is needed because a node’s radius ($H_{TD}/2$) is comparable to the success tolerance, while L2 is still discriminative at that range (section E.2).

3.3 BUDGET-CLOCKED EXPECTED HITTING TIME

A reachability critic $V(z, z_g, h) \in [0, 1]$, a five-member MLP ensemble over (z, z_g, h) with h on a grid of multiples of 5 up to $h_{\max} = 50$, estimates the probability that the goal is reached from z within h environment steps. It is trained on five terms:

- (i) hindsight: $y = \mathbf{1}_{h \geq \tau}$;
- (ii) cross-episode: $y = 0$ if $\|\psi(z) - \psi(z_g)\| \geq 1.25 u_{\max(h, 5)}$;
- (iii) imagined: predictor rollout under logged actions, labelled as in (i);
- (iv) Bellman: $q = \max_{m \leq M} (\bar{V}(\hat{z}'_m, z_g, h - 5) - \kappa \bar{\sigma}(\hat{z}'_m, z_g, h - 5))_{[0, 1]}$;
- (v) monotonicity: $\max(0, V(z, z_g, h) - V(z, z_g, h + 5))$.

Here τ is the first logged step with $\|\psi(z_{t+k}) - \psi(z_g)\| < u_2$, and (ii) keeps only cross-episode pairs that the TDR places beyond budget h . In (iv), $\hat{z}'_m = P(z, a_m)$ for M action blocks around and away from the logged one, and $\bar{V}, \bar{\sigma}$ are the mean and standard deviation of an EMA target ensemble. Terms (i)–(iv) enter by binary cross-entropy and (v) as a hinge. No label reads the simulator state, so training and inference are both pixel-only; on Push-T the TDR-derived τ agrees with the success predicate on 97.7% of draws.

Rather than the usual $-\log V$, we score an endpoint by its expected time to the goal, capped at the budget left after the plan being scored:

$$\text{ET}(\hat{z}_K, z_g) = 5 \sum_{h \in \{0, 5, \dots, h_{\text{rem}}\}} (1 - V(\hat{z}_K, z_g, h)), \quad h_{\text{rem}} = \min(h_{\max}, B - t - 5K), \quad (5)$$

where B is the episode budget and t the current step, kept by a *plan clock* that advances one horizon per CEM call; without it, the critic is asked about budget already spent. Feasible endpoints are ordered by arrival time, and infeasible ones sit at the cap. Both terms are standardised by median and interquartile range over each environment’s 300 CEM candidates (denoted $\widetilde{\cdot}$) and added:

$$c = \widetilde{c_{\text{base}}} + \beta \widetilde{\text{ET}}, \quad \beta = 1. \quad (6)$$

Algorithm 1 (section A) summarises one planning call.

What is trained. Only the TDR and the critic are trained (compute in section 4.4); E , P and CEM are LeWM’s released ones and the graph rules are GAS’s. Only $H_{TD} = u_{12}$ and $\lambda = u_{25}$ differ by environment, read off the TDR calibration; every other value is shared (section H).

4 EXPERIMENTS

4.1 SETUP

Environments and data. Push-T (??): the 18,685 expert episodes of LeWM’s released training file; success when agent and block position errors are below 20px and block angle error below $\pi/9$ (the environment’s own success check). Reacher (dm_control): 10,000 episodes of 201 frames; success when both joints are within 0.05 rad of the goal configuration (raw difference, no angle wrap; the dataset’s shoulder angle is unwrapped over ± 8 rad). OGBench Cube (?): LeWM’s dataset (?) of 10,000 episodes of 201 frames; success when the cube’s position is within 0.04 m of its target (CubeEnv.compute_successes, position only, no orientation or effector-pose requirement). Frozen LeWM checkpoints are the public ones for each environment.

Protocols. For a same-episode offset K , the goal is the frame K steps after the start on the same trajectory and the budget is $2K$ steps. For the cross-episode protocol the start and goal are uniformly random frames of two different episodes with no distance or reachability filter and a budget of one full episode. We use a fixed set of 50 tasks per protocol for every method and every seed. Each seed retrains the learned assets (TDR, graph, critic) and reseeds CEM sampling, so reported standard deviations include both sources of variation; L2 has no learned assets, and its seeds vary CEM sampling only.

Baselines. L2 is LeWM’s planner. TDR scores endpoints by $\|\psi(\hat{z}_K) - \psi(z_g)\|$ with no graph; CTG uses the graph cost-to-go $\min_v \|\psi(\hat{z}_K) - c_v\| + D_g[v]$ as a terminal value with no subgoal commitment. Random takes a uniform action from the env’s own action space at every step (no model, no planning); the floor L2 and Ours must clear (table 1). GCIQL is the offline RL baseline of ?, trained with stable-worldmodel’s released gciql configuration on the frozen LeWM latents, for 10 epochs per phase, with one training seed on Push-T only.

4.2 MAIN RESULT

On Push-T (table 1) the method is 2.8 points under L2 at same-ep 25 (88.4 vs 91.2), gains 29 points at same-ep 50 (70.0 vs 40.8), and is the strongest configuration in our study at same-ep 100 and on cross-episode goals (46.4 and 42.4 against 13.2 and 14.4). On Reacher, L2’s success rate holds up well for same-episode goals across horizons (77.2/90.8/82.8) but is poor cross-episode (29.2); the method matches this same-episode performance (76.0/98.0/97.6) and reaches 65.6% cross-episode, a +36-point gain. At same-ep 25 on Reacher the method is 1.2 points under L2. On Cube the method wins at every protocol, including same-ep 25 (62.0 vs 56.0) where Push-T and Reacher do not: +6.0 at same-ep 25, +15.2 at same-ep 50, +14.8 at same-ep 100, and +11.2 on cross-episode goals.

The GCIQL policy, trained on the same frozen latent and training episodes, reaches 42.0% at same-ep 25 on Push-T, above the random floor but 49 points below L2. It falls to 26.0 and 22.0 at same-ep 50 and 100 and solves 1 of 50 cross-episode tasks (2.0%). A reactive goal-conditioned policy learned offline from these data therefore does not substitute for planning with the world model, even where the goal lies within one planning horizon.

4.3 ABLATION LADDER

Table 2 adds one component at a time to Push-T, on the same task pool and seeds as table 1. The full ladder, including the β and H_{TD} sweeps, is in section B; the same rows on Cube are in section B.2.

Direction: the graph subgoal. A graph subgoal (b) lifts offset 50 and 100 over L2 (40.8 $\rightarrow$ 51.6%, 13.2 $\rightarrow$ 22.4%). The TDR metric alone (a) gets part of this, so the graph’s route adds to the metric, not just replaces L2’s geometry. At offset 50, L2’s closest approach is 32px but it drifts to a final error of 76px; with the subgoal, closest approach and final error are both 19px. The route matters

Table 1: Success rate (%). (A) Our evaluation on the fixed, held-out tasks, mean $\pm$ sd over five seeds. Within each environment, **bold** is the best mean and underlined means having a $\pm$ sd interval that overlaps the best’s. [†]One training seed with a deterministic policy, so no sd is reported; see section 4.1. (B) Numbers reported by ?. These were not run on our task set, which rejects pairs already solved at the start, so they are not directly comparable to (A). “–”: not reported.

(A) Our results					
Env	Method	same-ep 25	same-ep 50	same-ep 100	cross-ep
Push-T	Random	4.0 $\pm$ 1.3	0.4 $\pm$ 0.8	0.0 $\pm$ 0.0	0.4 $\pm$ 0.8
	GCIQL [†]	42.0	26.0	22.0	2.0
	L2 (LeWM)	91.2 $\pm$ 1.0	40.8 $\pm$ 3.7	13.2 $\pm$ 3.2	14.4 $\pm$ 3.9
	Ours	<u>88.4</u> $\pm$ 2.3	70.0 $\pm$ 2.2	46.4 $\pm$ 3.2	42.4 $\pm$ 3.2
Reacher	Random	8.0 $\pm$ 3.8	8.4 $\pm$ 0.8	9.2 $\pm$ 3.2	2.8 $\pm$ 1.0
	GCIQL [†]				
	L2 (LeWM)	77.2 $\pm$ 4.8	90.8 $\pm$ 1.0	82.8 $\pm$ 1.0	29.2 $\pm$ 2.4
	Ours	<u>76.0</u> $\pm$ 4.6	98.0 $\pm$ 1.3	97.6 $\pm$ 0.8	65.6 $\pm$ 2.0
Cube	Random	21.2 $\pm$ 3.0	12.8 $\pm$ 2.0	4.8 $\pm$ 4.1	3.6 $\pm$ 2.3
	GCIQL [†]				
	L2 (LeWM)	<u>56.0</u> $\pm$ 3.8	34.4 $\pm$ 3.2	35.6 $\pm$ 1.5	16.4 $\pm$ 1.5
	Ours	62.0 $\pm$ 4.7	49.6 $\pm$ 4.6	50.4 $\pm$ 3.2	27.6 $\pm$ 6.1

(B) Reported by ? (same-ep 25 only)							
Env	LeWM	PLDM	DINO-WM	GCBC	GCIQL	GCIVL	Ours
Push-T	96	78	74	75	20	33	88.4
Reacher	86	78	79	–	–	–	76.0
Cube	74	65	86	84	64	56	62.0

Table 2: Push-T ablation (success %, same tasks and seeds as table 1). Rows (a)–(c) build the graph subgoal without a critic; (d) adds the critic without a graph; (e)–(g) add a critic to the graph subgoal; (h) scores the subgoal with LeWM’s L2 instead of TDR distance. Full ladder in table 6. **bold**: best mean per column; underlined: $\pm$ sd interval overlaps the best’s.

	configuration	same-ep 25	same-ep 50	same-ep 100	cross-ep
	L2 (LeWM)	91.2 $\pm$ 1.0	40.8 $\pm$ 3.7	13.2 $\pm$ 3.2	14.4 $\pm$ 3.9
(a)	TDR terminal cost (no graph)	83.2 $\pm$ 4.1	48.4 $\pm$ 1.5	22.4 $\pm$ 5.3	14.0 $\pm$ 1.3
(b)	graph subgoal	80.8 $\pm$ 2.7	51.6 $\pm$ 2.3	22.4 $\pm$ 5.0	15.2 $\pm$ 3.0
(c)	(b) + final switch	<u>88.0</u> $\pm$ 2.5	62.4 $\pm$ 2.7	30.8 $\pm$ 6.5	17.2 $\pm$ 2.7
(d)	L2 + ET critic (no graph)	<u>89.2</u> $\pm$ 3.0	58.4 $\pm$ 5.1	<u>38.4</u> $\pm$ 7.1	31.6 $\pm$ 2.3
(e)	(b) + ET critic (no switch)	86.0 $\pm$ 3.4	<u>64.8</u> $\pm$ 3.0	<u>43.6</u> $\pm$ 5.0	35.2 $\pm$ 2.0
(f)	(c) + $-\log V$ critic	77.6 $\pm$ 1.5	<u>69.2</u> $\pm$ 2.4	47.2 $\pm$ 5.9	<u>34.4</u> $\pm$ 7.3
(g)	(c) + ET critic (ours)	<u>88.4</u> $\pm$ 2.3	70.0 $\pm$ 2.2	<u>46.4</u> $\pm$ 3.2	42.4 $\pm$ 3.2
(h)	(g), subgoal scored by L2	86.0 $\pm$ 2.2	<u>63.6</u> $\pm$ 4.8	<u>45.2</u> $\pm$ 3.5	<u>36.4</u> $\pm$ 2.9

more than how the subgoal is scored. In (h) the planner follows the same graph and subgoal node, but ranks candidates by LeWM’s L2 distance to the node’s medoid latent instead of TDR distance. It keeps most of the full method’s gain at offset 100 (45.2 vs. 46.4%) and loses about six points at offset 50 and cross-episode (63.6 vs. 70.0%, 36.4 vs. 42.4%).

Resolution: the final switch. Handing the last horizon back to c_{L2} (c) closes most of the same-ep 25 gap (80.8 $\rightarrow$ 88.0%) and lifts offset 50 and 100 further. On its own it leaves cross-episode unchanged (17.2%): resolution near the goal does not tell the planner which candidates can reach it. With the critic in place the switch still matters. Removing it (e vs. g) costs 2–7 points on every protocol, the most at cross-episode (35.2 vs. 42.4%). This is consistent with the node-radius floor: without the switch, the planner stops short of goals the critic has already judged reachable.

Table 3: Push-T success (%) in varying conditions: **(a)** Under loosened success criteria: Our method recovers more failures. **(b)** Cumulative success through each 50-step budget window; in brackets, the hazard rate: the share of tasks still unsolved before entering the window that are solved within it. Same-ep 100 evaluations are capped to a budget of 200 steps.

(a) Success threshold			(b) Success by budget window				
threshold	Ours	L2	window (steps)	same-ep 100		cross-ep	
				Ours	L2	Ours	L2
<i>same-ep 100</i>							
20 px/20°	46.4	13.2	0–50	8.8 (8.8)	8.8 (8.8)	14.8 (14.8)	6.0 (6.0)
25 px/25°	52.8	14.8	50–100	23.2 (15.8)	12.0 (3.5)	30.4 (18.3)	12.0 (6.4)
30 px/30°	58.0	16.0	100–150	36.4 (17.2)	12.4 (0.5)	36.4 (8.6)	13.2 (1.4)
<i>cross-ep</i>			150–200	46.4 (15.7)	13.2 (0.9)	38.8 (3.8)	14.0 (0.9)
20 px/20°	42.4	14.4	200–250	–	–	42.4 (5.9)	14.4 (0.5)
25 px/25°	51.2	19.2					
30 px/30°	57.6	22.8					

Reachability: the critic. The critic supplies most of the cross-episode gain. On top of L2 alone (d) it doubles cross-episode success (14.4 $\rightarrow$ 31.6%); on top of the graph (g) it reaches 42.4%. The graph therefore adds 8–12 points over the critic alone at offset 50, 100 and cross-episode, and the critic adds 8–25 points over the graph alone. Without the critic the graph finds a route but the block drifts once the approach begins (median final error 125 px vs. 26 px with the critic).

ET versus $-\log V$. The two critic costs tie at long range, but $-\log V$ (f) loses 11 points at offset 25. There the critic saturates (67% of the CEM population is feasible, versus 39–41% at offset 100 and cross-episode), and per-population standardisation turns that saturation into a full vote. The capped hitting time keeps ranking feasible candidates by how soon they arrive. With ET, a single $\beta = 1$ is best or within one standard deviation of the best on every protocol (table 6).

4.4 MECHANISTIC ANALYSIS

We give three explanations for the results in table 1. Table 3 compares Push-T success, at same-ep 100 and cross-episode, in varying conditions.

L2 ranks well only at short range; the graph degrades more gracefully. Scoring CEM’s own first-iteration elites against the privileged recovery-time oracle (section F), L2’s cost is the better ranker at same-ep 25 (Spearman 0.396 vs. 0.327 for the graph cost) but collapses beyond it, to 0.086 at same-ep 50 and -0.064 at same-ep 100 – statistically indistinguishable from chance. The graph cost degrades far more gracefully over the same range (0.327 $\rightarrow$ 0.203 $\rightarrow$ 0.191). This is the justification for the switch to L2 within the final horizon (equation (4)): L2 is the better signal at short range, and the graph is preferred at longer horizons where L2’s ranking fails.

Our failures are closer misses than L2’s. Loosening Push-T’s success threshold from its default 20 px/20° to 25 px/25° and 30 px/30° (table 3a) recovers far more of Our failures than L2’s: at same-ep 100, Ours rises 46.4 $\rightarrow$ 52.8 $\rightarrow$ 58.0% while L2 moves only 13.2 $\rightarrow$ 14.8 $\rightarrow$ 16.0%; at cross-episode, Ours rises 42.4 $\rightarrow$ 51.2 $\rightarrow$ 57.6% against L2’s 14.4 $\rightarrow$ 19.2 $\rightarrow$ 22.8%. The Ours-over-L2 gap widens rather than narrows under a looser criterion, so the advantage in table 1 is not an artifact of the strict threshold disproportionately penalizing Our near-misses – when Ours fails, it still remains closer to the goal than L2 does.

L2 stalls after its first replan; Ours keeps converging. Binning success by cumulative environment-step budget (table 3b) shows L2’s hazard rate – the fraction of still-unsolved tasks solved within each successive 50-step window – collapses to under 3.5% after the first window at both same-ep 100 and cross-episode: nearly all of L2’s successes come from its first two plans, and additional budget shows marginal improvement (cross-episode success rises only from 12.0% at 100 steps to 14.4% at 250). Our hazard rate instead stays between 8.6% and 18.3% through same-ep 100’s entire budget and cross-episode’s first three windows, so a substantial share of the tasks still unsolved keep getting solved by later replans. This explains how Our advantage in table 1

Table 4: Demonstration retrieval added to our method, with no other change (success %, same 50 held-out tasks and five seeds as table 1, mean $\pm$ sd). **Bold**: better mean of the two rows. Details in section C.

Env	Method	same-ep 25	same-ep 50	same-ep 100	cross-ep
Push-T	Ours	88.4 $\pm$ 2.3	70.0 $\pm$ 2.2	46.4 $\pm$ 3.2	42.4 $\pm$ 3.2
	Ours + retrieval	86.8 $\pm$ 2.0	66.8 $\pm$ 5.2	44.8 $\pm$ 4.7	42.4 $\pm$ 6.2
Reacher	Ours	76.0 $\pm$ 4.6	98.0 $\pm$ 1.3	97.6 $\pm$ 0.8	65.6 $\pm$ 2.0
	Ours + retrieval	62.0 $\pm$ 6.4	88.4 $\pm$ 2.7	94.0 $\pm$ 1.3	63.6 $\pm$ 2.3
Cube	Ours	62.0 $\pm$ 4.7	49.6 $\pm$ 4.6	50.4 $\pm$ 3.2	27.6 $\pm$ 6.1
	Ours + retrieval	86.4 $\pm$ 4.1	59.2 $\pm$ 2.4	78.8 $\pm$ 3.0	48.0 $\pm$ 2.8

Table 5: Push-T compute cost on one GTX 1080 Ti. Preparation is a one-off ~ 3.2 h, mostly critic training; planning with the graph and critic takes $1.6\times$ L2’s time per decision.

<i>One-off preparation (GPU-minutes, per seed)</i>	
Encode (shared across seeds)	49.6
TDR training (50k steps)	16.6
Graph construction	1.6
Critic training (60k steps)	123.0
Total (incl. shared encode)	190.8
<i>Per-decision planning time (s)</i>	
L2	1.14
Ours (graph subgoal + critic)	1.80
Ratio vs. L2	1.58 $\times$

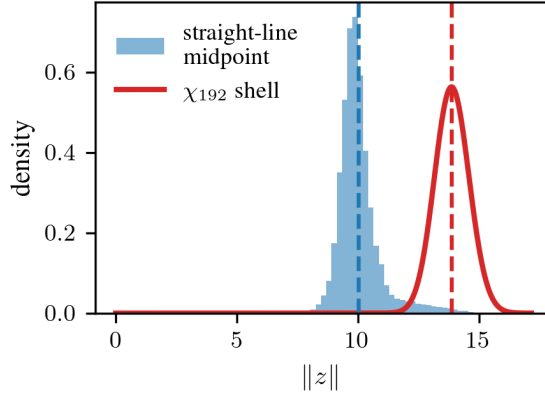


Figure 2: Straight-line midpoints leave the data distribution. Push-T, same-episode pairs $(z_{\text{start}}, z_{\text{goal}})$, 100 steps apart: radius of midpoints $\|\frac{1}{2}(z_{\text{start}} + z_{\text{goal}})\|$ (blue) vs. the χ_{192} shell that encoded frames lie in (red).

accumulates: it is able to maintain sustained progress at later steps, while L2 is incapable of doing so.

Demonstration retrieval helps Cube, but not Push-T or Reacher. CEM starts every plan from a zero-mean action distribution. Demonstration retrieval replaces that mean with a logged action block taken from the training data. At each planning call, we pick the dataset frame i minimising $\|\psi(z_i) - \psi(z_0)\| + \|\psi(z_{i+25}) - c_{v_\lambda}\|$: its segment starts near the current state and ends near the graph subgoal (or near the goal after the final switch). The 25 logged actions that follow i become CEM’s initial mean. Nothing else changes (section C). On Cube, retrieval adds 24.4, 9.6, 28.4 and 20.4 points to our method at the four protocols (table 4) and also lowers seed variance. The same change costs 1.6–3.2 points on Push-T and 2.0–14.0 points on Reacher. On Push-T the retrieved actions are accurate, but CEM does not need them to solve short pushes. On Reacher they are no better than random actions, because the two-joint arm reaches similar states through different joint motions. Cube’s grasp-then-carry motions are hard for CEM to find from a zero-mean start, and an imperfect demonstration is a large head start there. We therefore report retrieval as a Cube-specific option, not part of the method in table 1.

Compute. Table 5 reports preparation and planning cost; parameter counts are in table 11. Both learned additions are small next to the frozen encoder and predictor (18.03M): the TDR net used at planning time is 0.64M and the five-member critic 4.13M. The Dijkstra pass runs once per goal, not per candidate. Together, they add 58% to the planning time, with respect to L2.

5 WHY TERMINAL L2 FAILS BEYOND ONE HORIZON

The L2 rows of table 1 fall from 91.2% at offset 25 to 40.8% at 50, 13.2% at 100 and 14.4% cross-episode. We separate two causes.

Long open-loop rollouts compound predictor error. Widening LeWM’s default 2×25 -step rollouts naively to $2 \times H$ -step rollouts for longer offsets lowers L2’s success from 91.2% at offset 25 to 34.4% at offset 50 and 5.6% at offset 100 (section E.2). The predictor’s relative rollout error grows from 14.1% of the start–goal distance at $H = 25$ to 39.8% at $H = 100$. This failure is procedural: replanning every 25 steps minimizes accumulated predictor error, and every L2 result in this paper uses that cadence.

The terminal distance saturates. Replanning does not rescue the cost itself. In LeWM’s Push-T latent, two unrelated frames sit 19.6 apart on average. True same-episode start–goal distance is 17.1 at $H = 25$ but already 19.4 at $H = 50$ and 19.6 at $H = 100$ (section E.2): beyond one horizon, the goal is as far in L2 as a random frame, so terminal distance carries little information about which candidate is closer in time. This is consistent with the degradation of correlation between the L2 scores of candidates against the oracle observed in section 4.4.

L2 measures distance through a region the data does not occupy. The saturation has a geometric cause. section E.3 shows that, in high dimensions, an isotropic Gaussian puts almost all its mass on a thin shell of radius $\sigma\sqrt{d}$ ($= 13.88$) around the origin. figure 4 verifies this for LeWM’s Push-T latents. Two points on a shell are joined by a chord through its empty interior, while real trajectories stay on the shell. figure 2 shows that for same-episode pairs 100 steps apart, the straight-line midpoint $\frac{1}{2}(z_{\text{start}} + z_{\text{goal}})$ lies at radius 10.03, and 96.4% of them fall below the shell’s 1st percentile. Terminal L2 optimizes along this chord: it measures distance through states the data never visits, not the distance the dynamics must travel along the shell. The TDR and the graph (section 3.1) are built to follow the data instead.

6 LIMITATIONS

All results use one frozen model family; whether the same graph and critic help DINO-WM or PLDM is untested. SAGE (?) reports higher same-episode success than ours at long offsets with the same frozen model by training a subgoal generator on 400k expert windows; we do not claim the same-episode state of the art, and we expect a generator trained on single-trajectory windows to be unable to stitch, and thus not generalize adequately to cross-episode.

7 CONCLUSION

A frozen pixel world model’s planner fails past one horizon because its terminal cost cannot tell which candidates can still reach the goal. Giving it a data-supported target one horizon along a shortest path over the dataset, a budget-aware hitting-time estimate of what remains, and its own objective for the last horizon repairs mid-range, long-range and cross-episode goal reaching with nothing in the model or the planner retrained. The three pieces fix three different failures, and the per-pair diagnostics say why the last one is necessary: the graph knows a route for every cross-episode pair, and the planner cannot execute it without a reachability term.

AI USE STATEMENT

In this work, we used generative AI tools (an LLM-based coding assistant) to write and refactor implementation and analysis code, to help with literature search, and to edit text. We have not used generative AI tools to generate synthetic data, to develop the conceptual framework, or to formulate mathematical claims. We have reviewed all AI-assisted work: LLM-written code was read and tested by the authors, every reported number was produced by runs the authors launched and checked, and citations found with AI assistance were verified against the original papers. We take responsibility for the final content of this work, including text, claims or artifacts produced with the aid of generative AI. [TODO: Confirm the disclosure list with all authors.]

ETHICS STATEMENT

This work uses simulated benchmarks with public datasets and checkpoints and involves no human subjects or sensitive data.

REPRODUCIBILITY STATEMENT

We will release anonymised code as supplementary material. It covers dataset encoding, TDR training, graph construction, critic training, and CEM evaluation, together with the driver scripts used to produce every table. We will also release the fixed task-pool files and the trained TDR, graph, and critic for every seed. All experiments use the public LeWM checkpoints and datasets (?). Episodes that contain evaluation tasks are excluded from all learned-asset training and calibration. Section 4.1 describes the protocols and seeds, section A the planning procedure, section H all hyperparameters (including the unchanged CEM configuration), and table 5 compute costs. Live frames must be rendered with the MuJoCo version that produced the dataset (section D); we pin MuJoCo 3.5.0 and dm_control 1.0.37 for Reacher.

REFERENCES

- Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zhohus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khali-dov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025. URL <https://arxiv.org/abs/2506.09985>.
- Seungho Baek, Taegeon Park, Jongchan Park, Seungjun Oh, and Yusung Kim. Graph-assisted stitching for offline hierarchical reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2025. URL <https://arxiv.org/abs/2506.07744>.
- Jiaxin Bai and Jiaxuan Xiong. Temporal-distance JEPA: Plan-aware representation learning for latent world model predictive control. *arXiv preprint arXiv:2607.25337*, 2026. URL <https://arxiv.org/abs/2607.25337>.
- Niccolò Caselli, Francesco Massafra, Samuele Punzo, Salvatore Lo Sardo, Ippokratis Pantelidis, and Sathya Kamesh Bhethanabhotla. Mind the gap: Promises and pitfalls of hierarchical planning in LeWorldModel. *arXiv preprint arXiv:2607.12547*, 2026. URL <https://arxiv.org/abs/2607.12547>. WM@Booth 2026 Workshop on World Models.
- Letian Cheng, Qi Zhang, and Yisen Wang. SAGE: Subgoal-conditioned action generation for latent world model planning. *arXiv preprint arXiv:2607.17973*, 2026. URL <https://arxiv.org/abs/2607.17973>.
- Matthieu Destrade, Oumayma Bounou, Quentin Le Lidec, Jean Ponce, and Yann LeCun. Value-guided action planning with JEPA world models. *arXiv preprint arXiv:2601.00844*, 2026. URL <https://arxiv.org/abs/2601.00844>. World Modeling Workshop 2026.
- Scott Emmons, Ajay Jain, Michael Laskin, Thanard Kurutach, Pieter Abbeel, and Deepak Pathak. Sparse graphical memory for robust planning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. URL <https://arxiv.org/abs/2003.06417>.
- Benjamin Eysenbach, Ruslan Salakhutdinov, and Sergey Levine. Search on the replay buffer: Bridging planning and reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. URL <https://arxiv.org/abs/1906.05253>.
- Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International Conference on Machine Learning (ICML)*, 2019. URL <https://arxiv.org/abs/1811.04551>.

- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640:647–653, 2025. doi: 10.1038/s41586-025-08744-2. URL <https://doi.org/10.1038/s41586-025-08744-2>.
- Nicklas Hansen, Xiaolong Wang, and Hao Su. Temporal difference learning for model predictive control. In *International Conference on Machine Learning (ICML)*, 2022. URL <https://arxiv.org/abs/2203.04955>.
- Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2310.16828>.
- Jiaming Hu, Yan Zheng, Tian Wang, Florian Dubost, Alejandro Mottini, Junze Liu, Arvind Srinivasan, Kai Zhong, Kun Qian, Sharon Gao, and Qingjun Cui. SCALE: State-calibrated latent embeddings for JEPA planning in the right geometry. *arXiv preprint arXiv:2608.16287*, 2026. URL <https://arxiv.org/abs/2608.16287>.
- Beatrice Laurent and Pascal Massart. Adaptive estimation of a quadratic functional by model selection. *The Annals of Statistics*, 28(5):1302–1338, 2000.
- Liangyu Li, Shengzhi Wang, Libin Qiu, Mingliang Xiong, and Qingwen Liu. World model control by trajectory reachability metrics. *arXiv preprint arXiv:2605.22164*, 2026a. URL <https://arxiv.org/abs/2605.22164>.
- Wenyuan Li, Guang Li, Keisuke Maeda, Takahiro Ogawa, and Miki Haseyama. Predictive but not plannable: RC-aux for latent world models. *arXiv preprint arXiv:2605.07278*, 2026b. URL <https://arxiv.org/abs/2605.07278>.
- Kendall Lowrey, Aravind Rajeswaran, Sham Kakade, Emanuel Todorov, and Igor Mordatch. Plan online, learn offline: Efficient learning and exploration via model-based control. In *International Conference on Learning Representations (ICLR)*, 2019. URL <https://arxiv.org/abs/1811.01848>.
- Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorld-Model: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026. URL <https://arxiv.org/abs/2603.19312>.
- Sergi Masip, Jonathan Swinnen, Yutong Hu, Renaud Detry, and Tinne Tuytelaars. FF-JEPA: Long-horizon planning in world models with latent planners. *arXiv preprint arXiv:2606.09311*, 2026. URL <https://arxiv.org/abs/2606.09311>.
- Mitsuki Morita, Satoshi Yamamori, Satoshi Yagi, Norikazu Sugimoto, and Jun Morimoto. Goal-conditioned terminal value estimation for real-time and multi-task model predictive control. *arXiv preprint arXiv:2410.04929*, 2024. URL <https://arxiv.org/abs/2410.04929>.
- Evgenii Opyrshko, Junwei Quan, Claas Voelcker, Yilun Du, and Igor Gilitschenski. Test-time graph search for goal-conditioned reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2026. URL <https://arxiv.org/abs/2510.07257>.
- Seohong Park, Tobias Kreiman, and Sergey Levine. Foundation policies with Hilbert representations. In *International Conference on Machine Learning (ICML)*, 2024. URL <https://arxiv.org/abs/2402.15567>.
- Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. OGBench: Benchmarking offline goal-conditioned RL. In *International Conference on Learning Representations (ICLR)*, 2025. URL <https://arxiv.org/abs/2410.20092>.
- Nikolay Savinov, Alexey Dosovitskiy, and Vladlen Koltun. Semi-parametric topological memory for navigation. In *International Conference on Learning Representations (ICLR)*, 2018. URL <https://arxiv.org/abs/1803.00653>.
- Dikshant Shehmar, Matthew Schlegel, Matthew E. Taylor, and Marlos C. Machado. Laplacian representations for decision-time planning. In *International Conference on Machine Learning (ICML)*, 2026. URL <https://arxiv.org/abs/2602.05031>.

- Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL <https://arxiv.org/abs/2502.14819>.
- Basile Terver, Tsung-Yen Yang, Jean Ponce, Adrien Bardes, and Yann LeCun. What drives success in physical planning with joint-embedding predictive world models? *Transactions on Machine Learning Research (TMLR)*, 2026. URL <https://arxiv.org/abs/2512.24497>.
- Jiawei Wang, Yushen Zuo, Ke Rui, Yichun Feng, and Minglei Li. Decision-metric alignment in latent world models: Diagnostics and action-conditioned objectives for MPC planning. *arXiv preprint arXiv:2608.18746*, 2026a. URL <https://arxiv.org/abs/2608.18746>.
- Ying Wang, Oumayma Bounou, Gaoyue Zhou, Randall Balestriero, Tim G. J. Rudner, Yann LeCun, and Mengye Ren. Temporal straightening for latent planning. In *International Conference on Machine Learning (ICML)*, 2026b. URL <https://arxiv.org/abs/2603.12231>.
- J. G. Wendel. Note on the gamma function. *The American Mathematical Monthly*, 55(9):563–564, 1948.
- Wancong Zhang, Basile Terver, Artem Zhohus, Soham Chitnis, Harsh Sutaria, Mido Assran, Randall Balestriero, Amir Bar, Adrien Bardes, Yann LeCun, and Nicolas Ballas. Hierarchical planning with latent world models. *arXiv preprint arXiv:2604.03208*, 2026. URL <https://arxiv.org/abs/2604.03208>.
- Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. In *International Conference on Machine Learning (ICML)*, 2025. URL <https://arxiv.org/abs/2411.04983>.

A PLANNING ALGORITHM

Algorithm 1 One planning call of graph-assisted latent MPC

Require: frozen E, P ; CEM; TDR ψ ; graph (V_G, c_v, D_g) ; critic V ; budget B ; step t

- 1: $z_0 \leftarrow E(o_t)$; $h_{\text{rem}} \leftarrow \min(h_{\text{max}}, B - t - 5K)$
- 2: **if** $\|\psi(z_0) - \psi(z_g)\| \leq \lambda$ **then**
- 3: target $\leftarrow$ goal; base cost $\leftarrow c_{L2}$
- 4: **else**
- 5: $v^* \leftarrow \arg \min_{v: \|\psi(z_0) - c_v\| \leq H_{\text{TD}}} D_g[v] + \|\psi(z_0) - c_v\|$
- 6: walk the shortest path from v^* until cumulative TDR distance $\geq \lambda$; target $\leftarrow v_\lambda$; base cost $\leftarrow c_{\text{sub}}$ (goal and c_{L2} if the walk reaches the goal first)
- 7: **end if**
- 8: **for** each CEM iteration **do**
- 9: sample 300 action-block sequences; $\hat{z}_{1..K} \leftarrow P(\cdot)$
- 10: $c \leftarrow \text{base}(\hat{z}_K) + \beta \text{ET}(\hat{z}_K, z_g)$ with ET from equation (5)
- 11: refit the CEM Gaussian on the 30 lowest- c candidates
- 12: **end for**
- 13: execute the mean sequence for $5K$ steps; $t \leftarrow t + 5K$

B ABLATION LADDER

B.1 PUSH-T

Table 6: Full Push-T ablation ladder (success %, same tasks and seeds as table 1, mean $\pm$ sd over five seeds). Letters (a)–(h) match table 2. The lower blocks vary the critic weight β and the graph threshold H_{TD} (in TDR units) of the full method (g), with every other setting unchanged. The graph cost-to-go row uses the graph distance as a terminal value instead of a subgoal; its cost is $\min(\text{direct TDR distance, graph distance})$, so it departs from (a) only when the graph route is shorter. At same-ep 25 and cross-ep the two rows are separate runs that solve the same tasks on every seed.

	configuration	same-ep 25	same-ep 50	same-ep 100	cross-ep
	L2 (LeWM)	91.2 $\pm$ 1.0	40.8 $\pm$ 3.7	13.2 $\pm$ 3.2	14.4 $\pm$ 3.9
(a)	TDR terminal cost (no graph)	83.2 $\pm$ 4.1	48.4 $\pm$ 1.5	22.4 $\pm$ 5.3	14.0 $\pm$ 1.3
	graph cost-to-go (no subgoal)	83.2 $\pm$ 4.1	47.2 $\pm$ 1.6	21.6 $\pm$ 4.6	14.0 $\pm$ 1.3
(b)	graph subgoal	80.8 $\pm$ 2.7	51.6 $\pm$ 2.3	22.4 $\pm$ 5.0	15.2 $\pm$ 3.0
(c)	(b) + final switch	88.0 $\pm$ 2.5	62.4 $\pm$ 2.7	30.8 $\pm$ 6.5	17.2 $\pm$ 2.7
(d)	L2 + ET critic (no graph)	89.2 $\pm$ 3.0	58.4 $\pm$ 5.1	38.4 $\pm$ 7.1	31.6 $\pm$ 2.3
(e)	(b) + ET critic (no switch)	86.0 $\pm$ 3.4	64.8 $\pm$ 3.0	43.6 $\pm$ 5.0	35.2 $\pm$ 2.0
(f)	(c) + $-\log V$ critic	77.6 $\pm$ 1.5	69.2 $\pm$ 2.4	47.2 $\pm$ 5.9	34.4 $\pm$ 7.3
(g)	(c) + ET critic (Ours)	88.4 $\pm$ 2.3	70.0 $\pm$ 2.2	46.4 $\pm$ 3.2	42.4 $\pm$ 3.2
(h)	(g), subgoal scored by L2	86.0 $\pm$ 2.2	63.6 $\pm$ 4.8	45.2 $\pm$ 3.5	36.4 $\pm$ 2.9
	(g) with $\beta = 0.5$	88.4 $\pm$ 2.0	69.2 $\pm$ 2.7	43.2 $\pm$ 3.7	28.0 $\pm$ 3.8
	(g) with $\beta = 2$	87.6 $\pm$ 3.2	72.4 $\pm$ 4.1	42.8 $\pm$ 4.1	35.2 $\pm$ 3.5
	(g) with $\beta = 4$	86.8 $\pm$ 2.0	65.6 $\pm$ 2.3	48.4 $\pm$ 5.0	38.4 $\pm$ 3.4
	(g) with $H_{\text{TD}} = 4$	82.8 $\pm$ 2.0	64.0 $\pm$ 1.3	48.4 $\pm$ 6.1	35.2 $\pm$ 2.7
	(g) with $H_{\text{TD}} = 12$	89.6 $\pm$ 2.0	68.0 $\pm$ 3.8	48.0 $\pm$ 4.0	32.4 $\pm$ 6.1
	(g) with $H_{\text{TD}} = 16$	91.2 $\pm$ 2.4	64.4 $\pm$ 6.6	47.6 $\pm$ 3.4	36.8 $\pm$ 4.5

B.2 CUBE

Table 7 repeats the rows of table 2 on Cube. Tasks, seeds and protocols match table 1. The graph, subgoal and final-switch thresholds use Cube’s own calibration ($H_{\text{TD}} = 7.25$, lookahead and switch radius $u_{25} = 13.63$). Row (g) is the Cube row of table 1.

Table 7: Cube ablation (success %, 50 tasks, five seeds, mean $\pm$ sd). Rows as in table 2. **Bold**: best mean per column; underlined: $\pm$ sd interval overlaps the best’s.

	configuration	same-ep 25	same-ep 50	same-ep 100	cross-ep
	L2 (LeWM)	56.0 $\pm$ 3.8	34.4 $\pm$ 3.2	35.6 $\pm$ 1.5	16.4 $\pm$ 1.5
(a)	TDR terminal cost (no graph)	50.0 $\pm$ 1.8	48.8 $\pm$ 2.0	50.4 $\pm$ 1.5	36.8 $\pm$ 4.3
(b)	graph subgoal	49.2 $\pm$ 3.0	<u>46.8</u> $\pm$ 5.0	46.0 $\pm$ 3.6	<u>28.4</u> $\pm$ 4.1
(c)	(b) + final switch	54.4 $\pm$ 2.9	<u>44.8</u> $\pm$ 2.7	38.4 $\pm$ 3.7	<u>28.4</u> $\pm$ 4.3
(d)	L2 + ET critic (no graph)	64.0 $\pm$ 4.4	<u>45.2</u> $\pm$ 2.4	47.6 $\pm$ 3.7	17.6 $\pm$ 3.4
(e)	(b) + ET critic (no switch)	58.0 $\pm$ 1.8	<u>46.8</u> $\pm$ 5.2	56.8 $\pm$ 5.2	<u>28.4</u> $\pm$ 6.1
(f)	(c) + $-\log V$ critic	<u>56.8</u> $\pm$ 3.5	48.0 $\pm$ 2.2	39.6 $\pm$ 2.7	32.4 $\pm$ 4.8
(g)	(c) + ET critic (ours)	<u>62.0</u> $\pm$ 4.7	49.6 $\pm$ 4.6	<u>50.4</u> $\pm$ 3.2	<u>27.6</u> $\pm$ 6.1
(h)	(g), subgoal scored by L2	<u>63.6</u> $\pm$ 4.6	<u>42.4</u> $\pm$ 2.9	47.2 $\pm$ 1.6	26.4 $\pm$ 4.6

The ordering of components differs from Push-T. On Cube the TDR terminal cost alone (a) is the strongest configuration on cross-episode goals (36.8%), is within 0.8 points of the full method at same-ep 50 (48.8 vs. 49.6%), and ties it at same-ep 100 (50.4%). Adding the graph subgoal (b) does not improve on (a) in any protocol, and lowers cross-episode success by 8.4 points. The final switch (c) recovers 5.2 points at same-ep 25 but costs 7.6 points at same-ep 100. The ET critic is most useful at short range: on top of L2 alone (d) it adds 8.0 points at same-ep 25 and 10.8 at same-ep 50, but only 1.2 on cross-episode goals, whereas on Push-T the same row doubled cross-episode success. With the graph in place the critic changes cross-episode success by at most 4 points in either direction ((e)–(g) against (b)). The final switch also behaves differently: on Push-T it helps at every protocol, whereas on Cube removing it (e) gives the best result at same-ep 100 (56.8%).

On Cube, most of the full method’s gain over L2 comes from the learned temporal distance. The TDR terminal cost alone (a) accounts for nearly all of the gain at same-ep 50 and 100 (+14.4 and +14.8 points over L2, against +15.2 and +14.8 for the full method) and exceeds it on cross-episode goals. The critic supplies the gain at same-ep 25, where TDR alone falls below L2. The graph route and the final switch, which drive the Push-T results, add little on Cube. The contribution of each component is therefore environment-dependent, and the component-wise conclusions of section 4.3 are specific to Push-T. The combined method is nonetheless the most robust configuration: it is the only row that is best or within one standard deviation of the best on every protocol in both table 2 and table 7.

C DEMONSTRATION RETRIEVAL

Method. CEM’s first population is sampled from $\mathcal{N}(\mathbf{0}, I)$ in z-scored action space, so every plan starts from “do nothing”. Retrieval uses the dataset as an action library to choose a better starting mean. Let $\mathcal{T}$ be the training frames whose episode has at least $L = 25$ steps left (five 5-action blocks, one planning horizon). At each planning call with current latent z_0 and target t , where $t = c_{v_\lambda}$ while the graph subgoal is active and $t = \psi(z_g)$ after the final switch (equation (4)), we select

$$i^* = \arg \min_{i \in \mathcal{T}} \|\psi(z_i) - \psi(z_0)\| + \|\psi(z_{i+L}) - t\|, \quad (7)$$

by exact nearest-neighbour search on the GPU. The logged actions $a_{i^*}, \dots, a_{i^*+L-1}$ are z-scored with the same normaliser the predictor uses, reshaped into five action blocks, and passed to CEM as its initial mean. The variance, number of samples, elites, iterations, cost and replanning interval are unchanged.

Motivation. Cube’s action space is five-dimensional, against two dimensions on Push-T and Reacher. With a fixed budget of 300 samples per iteration, a zero-mean CEM population covers this larger space far more sparsely, so good action sequences are less likely to be sampled at all. Retrieval addresses this by starting the search from a demonstrated action block instead of from zero. We evaluate it on all five seeds and all four protocols, for all three environments (table 4); the retrieval flag is the only difference from our method’s runs.

Results. On Cube, retrieval wins in every protocol and on every seed at same-ep 25, same-ep 100 and cross-episode. The gain is smallest at same-ep 50 (+9.6) and largest at same-ep 100 (+28.4) and cross-episode (+20.4). Its seed-to-seed sd is lower than our method’s in every protocol (e.g. 2.8 vs. 6.1 on cross-episode). On Push-T it loses 1.6–3.2 points in the same-episode protocols and ties on cross-episode, with higher variance. On Reacher it loses in every protocol, most at same-ep 25 (−14.0) and least on cross-episode (−2.0). Retrieval therefore does not help on Push-T or Reacher, likely because in their two-dimensional action spaces the CEM population alone already covers the space adequately, so a demonstrated starting mean adds little and can bias the search.

D LIVE RENDERING MUST MATCH THE DATASET

The frozen encoder is sensitive to renderer versions. Reacher frames rendered with MuJoCo 3.12 lack the floor checker texture of the dataset’s MuJoCo 3.5 frames; the pixel difference is 18/255 on average and moves the latent by a median of 15 units, about one 25-step gap, so the terminal cost never reaches zero even with the arm at the goal. This alone took LeWM’s own planner from 87% to 65% at offset 25.

E WHERE THE LATENT-L2 OBJECTIVE FAILS

This section traces one failure mode of terminal latent L2. Section E.1 states two properties of high-dimensional isotropic Gaussians that the rest of the section uses. Section E.2 measures how far apart start and goal latents sit as the horizon grows, and section E.3 shows why a straight line between two such latents leaves the region the encoder populates. Section F checks the same failure inside CEM’s own candidate ranking.

E.1 CHARACTERISTICS OF HIGH-DIMENSIONAL GAUSSIANS

Let $z \sim \mathcal{N}(\mu, \sigma^2 I_d)$, let χ_d denote the Euclidean norm of a standard Gaussian vector in $\mathbb{R}^d$, and write $m_d = \mathbb{E}[\chi_d] = \sqrt{2} \Gamma(\frac{d+1}{2}) / \Gamma(\frac{d}{2})$ for the mean of a chi distribution with d degrees of freedom. By Wendel’s inequality for ratios of Gamma functions (?),

$$\frac{d}{\sqrt{d+1}} \leq m_d \leq \sqrt{d}. \quad (8)$$

Lemma 1 (Distance between random pairs). *Let z_1, z_2 be independent draws of $\mathcal{N}(\mu, \sigma^2 I_d)$. Then $\|z_1 - z_2\| \sim \sqrt{2} \sigma \chi_d$, so*

$$\mathbb{E}\|z_1 - z_2\| = \sqrt{2} \sigma m_d \in \left[\sqrt{2} \sigma \frac{d}{\sqrt{d+1}}, \sqrt{2d} \sigma \right], \quad \text{Var} \|z_1 - z_2\| = 2\sigma^2(d - m_d^2) \leq 2\sigma^2.$$

In particular, $\mathbb{E}\|z_1 - z_2\| = \sqrt{2} \mathbb{E}\|z - \mu\|$.

Proof. $z_1 - z_2$ is Gaussian with mean 0 and covariance $2\sigma^2 I_d$, so $\|z_1 - z_2\|/(\sqrt{2}\sigma)$ is the norm of a standard normal vector in $\mathbb{R}^d$, which is χ_d -distributed. The mean follows from the definition of m_d and equation (8). For the variance, $\mathbb{E}[\chi_d^2] = d$, so $\text{Var}[\chi_d] = d - m_d^2 \leq d - d^2/(d+1) = d/(d+1) < 1$ by the lower bound in equation (8). The last claim holds because $\|z - \mu\| \sim \sigma \chi_d$. $\square$

The typical distance between unrelated points therefore grows as $\sigma\sqrt{2d}$, while its standard deviation stays below $\sqrt{2}\sigma$. At $d = 192$ the ratio of standard deviation to mean is at most $\sqrt{d+1}/d < 7.3\%$: nearly all unrelated pairs are about equally far apart.

Proposition 2 (Gaussian shell). *Let $r = \|z - \mu\|$ with $z \sim \mathcal{N}(\mu, \sigma^2 I_d)$. Then $\mathbb{E}[r] = \sigma m_d \in [\sigma d/\sqrt{d+1}, \sigma\sqrt{d}]$ and $\text{Var}[r] \leq \sigma^2$. Moreover, for every $\varepsilon \in (0, 1]$,*

$$\Pr[r \leq (1 - \varepsilon) \sigma\sqrt{d}] \leq \exp(-d\varepsilon^2/4).$$

Proof. $r/\sigma \sim \chi_d$, so the moments follow as in lemma 1. For the tail, the lower-tail bound of ? gives $\Pr[\chi_d^2 \leq d - 2\sqrt{dx}] \leq e^{-x}$ for all $x > 0$. The event $r \leq (1 - \varepsilon)\sigma\sqrt{d}$ is the event $\chi_d^2 \leq d(1 - \varepsilon)^2$. Choose x with $d - 2\sqrt{dx} = d(1 - \varepsilon)^2$, that is $\sqrt{x} = \sqrt{d}(2\varepsilon - \varepsilon^2)/2$. Since $2\varepsilon - \varepsilon^2 \geq \varepsilon$ on $(0, 1]$, $x \geq d\varepsilon^2/4$, and the bound follows. $\square$

Lemma 2 says that the mass of a high-dimensional isotropic Gaussian lies in a thin shell of radius about $\sigma\sqrt{d}$ and width $O(\sigma)$, and that the interior of the ball is exponentially unlikely. One consequence matters for planning. If z_1, z_2 are independent, their midpoint satisfies $\frac{1}{2}(z_1 + z_2) - \mu \sim \mathcal{N}(0, \frac{\sigma^2}{2}I_d)$, so by lemma 2 it concentrates at radius about $\sigma\sqrt{d}/2$, a factor $1/\sqrt{2}$ inside the shell. A straight line between two typical points therefore passes through a region that the distribution itself almost never visits.

E.2 PREDICTOR ROLLOUT ERROR BY HORIZON

Rolling the frozen predictor forward under real logged actions and comparing to the true encoded endpoint, Push-T’s latent rollout error grows with horizon but stays well below the scale of two unrelated frames (mean 19.6) even at $H = 100$ (table 8 and figure 3): the predictor degrades gracefully rather than collapsing to an uninformative guess. The random-pair scale agrees with lemma 1: with the measured mean radius 13.88 (section E.3), the lemma predicts a mean random-pair distance of $\sqrt{2} \times 13.88 \approx 19.63$, against the measured 19.6.

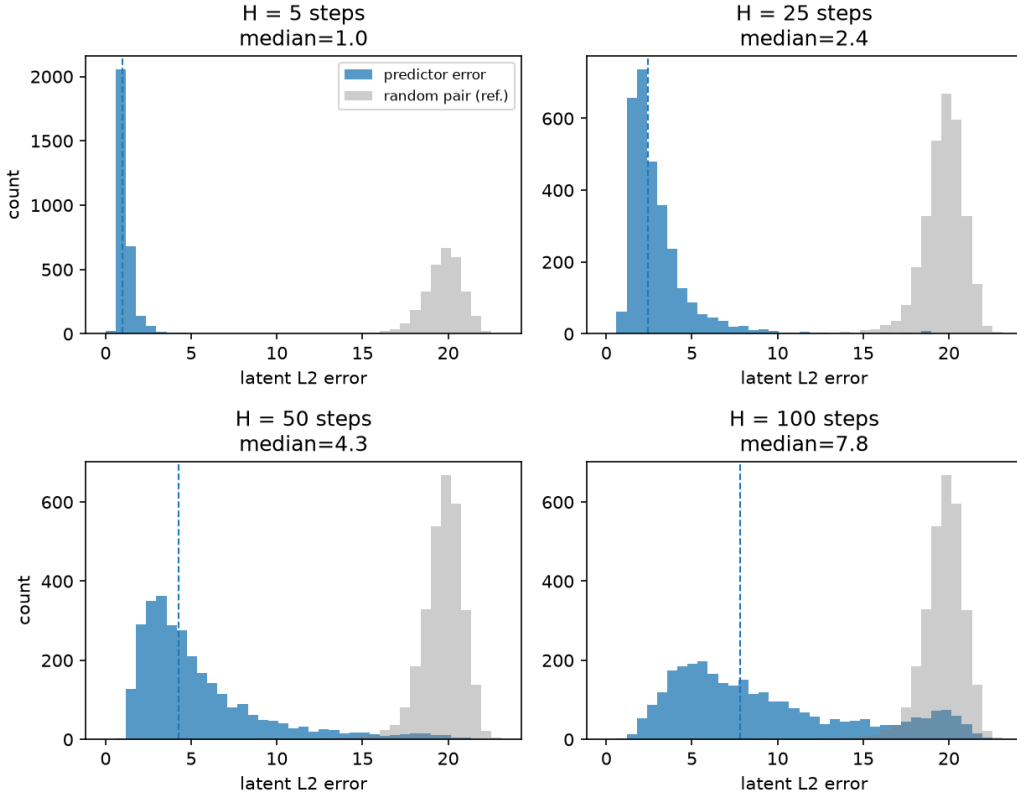


Figure 3: Push-T predictor latent rollout error (blue) vs. horizon, against the random-pair reference distribution (gray).

Task difficulty: how far apart are the actual start/goal pairs? The random-pair reference above (mean 19.6) is the scale at which two frames carry no information about each other. Sampling same-episode (start, start+ H) pairs the same way figure 3 samples its rollout starts, but reading off the true encoded distance $\|z_{\text{start}} - z_{\text{start}+H}\|$ instead of rolling the predictor forward (table 8), shows that $H = 50/H = 100$ are close to indistinguishable from random-pairs (19.4, 19.6) – consistent with section 5’s Spearman collapse past $H=50$. Relative to that task scale, the predictor’s own rollout error is at 14.1% of the gap at $H = 25$ and grows to 39.8% by $H = 100$ – a sizeable fraction of the

very distance CEM is trying to optimize over, consistent with L2’s success falling as its plans reach further ahead (table 9).

Table 8: Push-T predictor error compared with task distance. For each horizon H we report the latent L2 error of an H -step predictor rollout and the true distance between a start frame and the frame H steps later in the same episode (median $\pm$ std,). Relative error is the ratio of the two medians. For reference, two random frames lie 19.6 apart on average, matching the Gaussian prediction (section E.3).

	$H = 5$	25	50	100
predictor error	1.00 ± 0.56	2.41 ± 2.09	4.28 ± 3.82	7.80 ± 5.35
start→goal distance	5.29 ± 2.76	17.06 ± 4.19	19.45 ± 3.29	19.60 ± 2.43
relative error (%)	18.9	14.1	22.0	39.8

A related question concerns the planner itself: how far ahead should the L2 baseline plan before replanning? Table 9 varies the plan length over 5 steps (a single action block), 25 steps (the default), 50 and 100 steps. The CEM horizon is set equal to the executed chunk in every case, so the planner executes exactly the span it evaluated. Budgets, tasks and seeds follow table 1.

Table 9: Push-T L2 baseline versus plan length. The default is 25 steps. Success (%), mean $\pm$ sd over 5 CEM seeds

protocol (budget)	plan length (env steps)			
	5	25 (default)	50	100
same-ep 25 (50)	64.0 ± 1.8	91.2 ± 1.0	–	–
same-ep 50 (100)	19.2 ± 2.7	40.8 ± 3.7	34.4 ± 2.7	–
same-ep 100 (200)	1.2 ± 1.6	13.2 ± 3.2	12.8 ± 2.4	5.6 ± 2.3
cross-ep (250)	4.4 ± 1.5	14.4 ± 3.9	15.2 ± 4.8	4.8 ± 2.0

Success is non-monotonic in plan length, with the best results at 25–50 steps. Committing to 100 steps reduces L2’s success by 7.6 points at same-ep 100 and 9.6 points at cross-ep, which is consistent with predictor error compounding over a long open-loop chunk. Shorter plans perform worse still. With a single-block lookahead the planner becomes greedy and cannot plan the multi-step approach required before the block moves. At long range, the 50-step setting is within seed variation of the default. The default 25-step plan length is at or near the best setting for L2, and the gaps reported in table 1 do not arise from the choice of plan lengths.

E.3 WHY A STRAIGHT LATENT LINE GOES OUT OF DISTRIBUTION

Section E.2 shows the predictor itself degrades gracefully with horizon; this section asks why the terminal L2 objective built on top of it still fails past $H \approx 50$ (elite-ranking Spearman $0.396 \rightarrow 0.086$, section F). By lemma 2, the mass of a high-dimensional isotropic Gaussian lies on a thin shell of radius about $\sigma\sqrt{d}$. If LeWM’s Push-T latent cloud approximates such a shell, a straight line drawn between two points *on* the shell passes through the ball’s interior, which the shell distribution assigns almost no mass – exactly the regime a z -space-L2 CEM plan implicitly cuts through when it scores a candidate purely by terminal distance to a goal many steps away.

The latent cloud is close to an isotropic shell. LeWM’s SIGReg term regularises the encoder towards an isotropic Gaussian, so the theory above should apply to its latents directly. We test this on encoded Push-T frames ($d = 192$). The latents are centred: their empirical mean has norm 0.59, against a typical latent norm of 13.9, so we take $\mu = 0$. With $\sigma^2 = \text{tr}(\Sigma)/d$ the average per-dimension variance, the theory predicts that $\|z\|$ follows $\sigma\chi_d$, with mean 13.88. The encoded frames match this almost exactly (empirical mean 13.89, figure 4).

A straight-line midpoint is far off the shell; the real trajectory is not. For 200,000 same-episode frame pairs 100 steps apart (the same-ep-100 horizon), both endpoints sit on the shell as expected (start mean 14.15, goal mean 13.60), and so does the actual logged frame at the trajectory’s

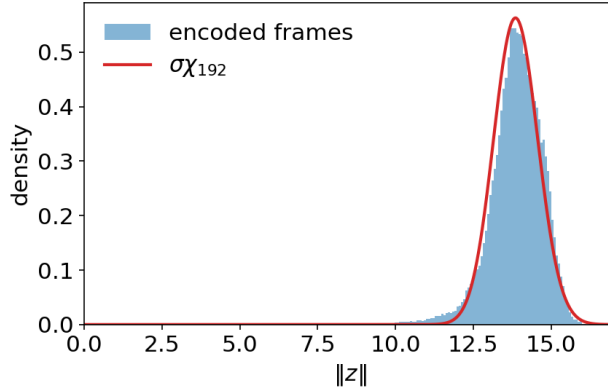


Figure 4: Push-T: norm $\|z\|$ of encoded frames (histogram of 200,000 of the 2.3M frames) vs. the $\sigma\chi_{192}$ distribution predicted for a zero-mean isotropic Gaussian with the same per-dimension variance ($\sigma = 1.003$). Empirical mean 13.89 (sd 0.82); predicted mean 13.88 (sd 0.71).

true midpoint (mean 13.81). The linear interpolation $\frac{1}{2}(z_{\text{start}} + z_{\text{goal}})$ does not: its mean radius is 10.03, 5.42 shell- σ below the shell mean, and 96.4% of the 200,000 interpolated midpoints fall below the shell’s own 1st percentile radius (figure 2 in section 5). The two populations do not overlap. This isolates the effect to interpolation itself, not to trajectory midpoints being unusual in general: the real dynamics stay on the shell, curving around it rather than cutting through its interior, which is exactly the structure a graph built in a temporal-distance representation is meant to follow instead of a raw latent straight line (sections 3 and 4.2).

F CANDIDATE RANKING WITHIN CEM’S OWN ELITES

The ranking comparison in section 4.4 is performed on candidates drawn from the planner’s own search rather than on a fixed diagnostic pool. This appendix defines the procedure.

Candidate set. For each task i , with start state $s_0^{(i)}$ and goal observation $o_g^{(i)}$, we run the first CEM solve of OUR and stop after iteration 1. At this iteration both costs are evaluated on the same population of $N = 300$ action sequences, sampled from CEM’s default initialisation $\mathcal{N}(\mathbf{0}, I)$ in normalised action space. No elite selection has yet shaped this population, so it is unbiased with respect to either cost. Let $\mathcal{E}_i = \{a^{(i,1)}, \dots, a^{(i,K)}\}$ be the $K = 30$ elites selected by OUR’s cost. Each $a^{(i,k)}$ is a 25-step sequence of raw actions, obtained by de-normalising and clipping to the action bounds.

Costs. For every $a \in \mathcal{E}_i$, the frozen predictor produces a single imagined rollout. We evaluate two costs on this rollout: OUR’s planning cost $c_{\text{OUR}}(a)$ (the complete objective of section 3), and LeWM’s terminal cost $c_{\text{L2}}(a) = \|\hat{z}_H(a) - z_g\|_2^2$. The two costs therefore differ only in the criterion, not in the predicted trajectory.

Oracle label. Each elite is executed in the simulator from $s_0^{(i)}$, reaching the true state $s^{(i,k)}$. From $s^{(i,k)}$, a privileged CEM planner with access to the true simulator (population 64, 5 iterations, 2 independent trials) searches for the smallest number of steps after which the environment’s success predicate holds for goal $o_g^{(i)}$. The recovery time is

$$T^{(i,k)} = \min \{ t \leq T_{\text{max}} : \text{the oracle reaches the goal in } t \text{ steps} \}, \quad (9)$$

where the minimum is taken over trials, and $T^{(i,k)} = T_{\text{max}} + 1$ if no trial succeeds within the cap $T_{\text{max}} = 100$. Such labels are right-censored: they indicate that no plan was found, not that recovery is impossible. Lower T indicates a better candidate. The oracle uses the simulator state and is not available to either planner.

Statistic. For each task we compute the Spearman rank correlation $\rho_i(c) = \text{Spearman}(\{c(a^{(i,k)})\}_{k=1}^K, \{T^{(i,k)}\}_{k=1}^K)$ for $c \in \{c_{\text{OUR}}, c_{\text{L2}}\}$. A positive value means that the cost assigns lower values to candidates that recover faster. Tasks on which all K labels are equal (typically all censored) have undefined ρ_i and are excluded. We report the mean $\bar{\rho}(c) = |\mathcal{I}|^{-1} \sum_{i \in \mathcal{I}} \rho_i(c)$ over the set $\mathcal{I}$ of remaining tasks, with 95% confidence intervals from a bootstrap over tasks.

Setup. We use 25 tasks from each task pool, CEM seed 0, and learned-asset seed 0. Because $\mathcal{E}_i$ is selected by c_{OUR} , the comparison measures how well each cost orders the candidates that OUR would retain. It does not measure how well each cost ranks the full population.

Table 10: Mean per-task Spearman correlation $\bar{\rho}$ between each cost and the oracle recovery time T over OUR’s $K = 30$ elites at CEM iteration 1. $|\mathcal{I}|$ is the number of tasks with non-degenerate labels. At same-ep 100, 14 of 25 tasks were fully censored even with a larger oracle cap.

protocol	$ \mathcal{I} $	$\bar{\rho}(c_{\text{OUR}})$	$\bar{\rho}(c_{\text{L2}})$
same-ep 25	25	0.327	0.396
same-ep 50	24	0.203	0.086
same-ep 100	11	0.191	−0.064

Result. At same-ep 25, c_{L2} has the higher mean correlation. At same-ep 50 and 100, c_{OUR} has the higher correlation. The L2 ranking at these horizons is not distinguishable from chance. The crossover matches the short-range/long-range pattern of closed-loop success in table 1, here measured on the candidate ranking of a single CEM iteration.

G PARAMETER COUNTS

Table 11 lists the parameters of the frozen LeWM stack and of the two components we train. Only ψ_0 of the TDR’s two networks is used at planning time.

Table 11: Push-T parameter counts. All LeWM modules are frozen.

LeWM encoder (ViT-tiny)	5.50M
LeWM predictor	10.79M
LeWM action encoder	0.16M
LeWM projector (encoder head)	0.80M
LeWM predictor projector	0.80M
LeWM total (frozen)	18.04M
TDR (used, ψ_0 only)	0.64M
TDR (full 2-net ensemble)	1.29M
Critic (per member)	0.83M
Critic (5-member ensemble)	4.13M

H HYPERPARAMETERS

Table 12 lists hyperparameters shared across Push-T, Reacher, and Cube; table 13 lists ones that are environment specific. The per environment scalars in table 13 are calibrated directly from the statistics inferred from the dataset, and are not tuned per environment. CEM uses LeWM’s released evaluation configuration in every environment; the LeWM paper reports 10 iterations outside Push-T.

Table 12: Hyperparameters shared across all three environments. TDR and graph follow section 3.1; critic follows section 3.3; CEM and the criterion follow equation (6) and algorithm 1.

component	hyperparameter	value
TDR ψ	architecture	3×512 LayerNorm+GELU MLP, linear head
	embedding dim	32
	goal relabelling	Geom($1 - \gamma$) future w.p. 0.625, else random
	value networks	2 (TD target: min)
	expectile τ	0.99
	discount γ	0.99
	training steps	50,000
	batch size	1024
	learning rate	3×10^{-4}
Graph	TE threshold θ_{TE}	0.9
	clustering radius	$H_{TD}/2$
	goal attachment radius	$\max(H_{TD}, 1.2 d_{\min})$
Critic V	ensemble members	5
	architecture (per member)	3×512 MLP + SiLU, linear head
	max horizon $h_{\max}$	50 env steps
	horizon grid	multiples of 5, $0-h_{\max}$
	training steps	60,000
	hindsight radius for τ	u_2
	Bellman candidates M	8 (1 logged, 4 perturbed, 3 random)
	Bellman pessimism κ	1.0
	loss weights ($\lambda_{\text{Bellman}}, \lambda_{\text{mono}}$)	1.0, 1.0
	learning rate	3×10^{-4}
	EMA target τ	0.995
	cross-episode negative gate	$\ \psi(t) - \psi(g)\ \geq 1.25 \times d_{\text{cal}}(h)$
CEM (LeWM, unchanged)	population / iterations / elites	300 / 30 / 30
	horizon K	5 blocks $\Rightarrow$ 25 env steps
	replan interval (receding horizon)	25 env steps
Criterion (equation (6))	critic weight β	1
	critic cost	ET (budget-capped expected hitting time)
	final-phase metric	LeWM’s own z-space L2 to the goal

Table 13: Environment-specific hyperparameters. u_k denotes the calibrated TDR distance of a k -env-step transition (median over training transitions), read from each environment’s own TDR calibration. $\lambda = u_{25}$ and $H_{TD} = u_{12}$ in every environment. Every other hyperparameter is identical across environments.

	Push-T	Reacher	Cube
H_{TD} (TDR units)	7.59	3.72	7.25
$\lambda = u_{25}$ (TDR units)	13.53	5.83	13.63
graph nodes	20.5k	535	16k